Warsaw Hackathon Challenge FAQ

During our hackathon you will be able to experience real world challenges that roboticists face on a daily basis. We have prepared 3 tracks with team limits for each track.

1. YAM Arm challenge (7 teams):

One of the factories asked us if the robots would be able to plug the wire rope into a rubber cable. This is a real use-case that currently humans perform.

The challenge is about training a policy that will achieve the highest success ratio. **You can train any model you like in any way you like!** In this track you will work with the YAM arms (https://i2rt.com/products/yam-6-dof-arm). We have provided for you the fork of Lerobot at:

Data

We have collected for you a dataset recorded by the professional tele-operators (around 10 hours) with all successful and repetitive trajectories. The task specific data is located at:

```
/common/data/yam_wire_data
```

Policy deployment on YAM arms

Your policies will be deployed on our **local GPU server**, which is physically connected to **bimanual YAM arms**.

For **safety reasons**, teams are **not allowed to connect directly** to the robot hardware from their own machines.

We prepared a repository which you can use as a base - lerobot_hackathon, but feel free to implement things your own way as long as they are going to be compatible with our setup described below.

Architecture Overview

- The GPU server runs a **robot client** that interfaces directly with the YAM arms.
- Your team provides a **policy inference server** which can load up your model.
- The robot client sends observations to your policy server and executes the returned actions.

The robot client that we run on our machine is:

src/lerobot/async_inference/robot_client.py lute-corp/lerobot_hackathon
main

And you can leverage the existing policy server:

src/lerobot/async_inference/policy_server.py lute-corp/lerobot_hackathon
main

Your inference server can run:

- **On your local machine** (if you have sufficient compute), or
- **On our server**, packaged as a Docker image (recommended).

▼ **Deployment Steps**

1. **Docker Image Setup**
   - You provide a Docker image for policy inference.
   - Model weights can be **baked into the image** or **mounted as a volume**.
2. **Policy Server Launch**
   - We start your policy inference server: https://github.com/lute-corp/lerobot_hackathon/blob/main/src/lerobot/async_inference/policy_server.py
   - We run the container with port exposed to the host system so that it can communicate with the robot client
3. **We run the robot client, which:**
   - Relays real robot observations to your policy
   - Executes the actions returned by your policy server
4. **Safety Check (visualization-only mode)**
   - **First run is always done with** `-viz_only=true`
   - Actions are played in the **visualizer only**, not on the real robot
   - Observations still come from the real robot
   - This way we verify that the policy behaves correctly
5. **Live Deployment**
   - Once behavior looks correct in the visualizer, we deploy the policy on the **real YAM** arms

▶ **Getting Help**

2. SO Arm Challenge (5 teams)

In this track you will be working with SO arms (could be both SO100 or SO101). Given the constraints of the hardware you can try to solve the task in any way you like (yes, you can cheat!). Go ahead with printing additional stuff, collect more data, change the task setup. Everything is allowed. You will be given equipment to work completely on your own.